CS336 作业 4：语言建模数据过滤

版本 26.0.1 · CS336 课程组 · 2026 年春季

译文说明：完整翻译原作业说明，保留题目标识、分值、提交要求、数据路径与测试命令。代码和输出保留原文；本文件不包含习题解答。

1 作业概述

本次作业将带你过滤网页抓取数据，构建语言建模训练集。

你将实现的内容

1. 将 Common Crawl HTML 转换为文本。

2. 以多种方式过滤提取的文本，例如处理有害内容、个人身份信息等。

3. 对训练数据去重。

你将运行的实验

1. 在不同数据集上训练语言模型，更好地理解具体数据处理决策对性能的影响。

代码结构

全部代码与说明位于 https://github.com/stanford-cs336/assignment4-data 。请用 git clone 克隆仓库，收到更新通知后用 git pull 获取最新版。

1. cs336_basics/*：包含作业 1 模型的训练代码。做了少量优化，将一些手写组件换为 PyTorch 原生版本，例如内置交叉熵内核。还提供支持多 GPU 分布式数据并行的训练脚本，用于在过滤后的数据上训练。

2. cs336_data/*：编写本作业代码的位置。模块包含起始代码与待实现的占位内容。

3. tests/*.py：必须通过的全部测试。测试调用 tests/adapters.py 中定义的钩子，你要实现适配器接入自己的代码。可新增或修改测试以辅助调试，但最终必须通过原始测试套件。

4. README.md：目录结构详情和基本环境配置说明。

如何提交

向 Gradescope 提交 writeup.pdf，回答全部书面问题并正式排版；另提交 code.zip，包含你编写的全部代码。

2 过滤 Common Crawl

虽然大型语言模型主要使用互联网数据训练，多数研究者并不会自己编写网页爬虫，而是使用公开抓取数据。最流行的来源是非营利组织 Common Crawl，它免费提供网页语料，原文称其覆盖“17 年间超过 2500 亿个页面”。[脚注 1：https://commoncrawl.org/]

但要把 Common Crawl（CC）转储数据变成可用的语言模型训练数据，需要大量处理。例如网页原始内容是 HTML，需要提取文本；许多页面质量低、完全或近似重复、包含有害内容或敏感信息，因此可能要删除页面，或从数据集中移除不需要的部分。本作业要搭建执行其中若干步骤的流水线，将原始互联网数据转为可用训练集。

2.1 查看数据

动手实现前，先查看原始数据、形成直观认识总是有益的。CC 提供三种格式：

• WARC（Web ARChive format）：原始 CC 数据，包含页面 ID、URL、元数据、HTTP 请求信息，例如请求日期时间、服务器 IP，以及 HTML 等原始页面内容。

• WAT（Web Archive Transformation）：从 WARC 提取的高层元数据，以 JSON 对象保存。例如 HTML 页面中的链接列表、页面标题。

• WET（Web Extracted Text）：从原始 HTML 中提取的纯文本。

下面查看一个 WARC 及其对应 WET。它们来自 2026 年的一次 Common Crawl 抓取，下载命令如下：

```
# Download a sample WARC file.
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/
warc/CC-MAIN-20260310152245-20260310182245-00000.warc.gz
# Download its corresponding WET file.
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/
wet/CC-MAIN-20260310152245-20260310182245-00000.warc.wet.gz
```

原文第 2 页

本地文件位于 /shared-data/CC/example.warc.gz 和 /shared-data/CC/example.warc.wet.gz。

警告：这些文件包含完全未过滤的互联网页面，可能有大量潜在有害内容。如果遇到不想阅读的文档，可直接继续滚动并查看其他文档。

题目（look_at_cc）：查看 Common Crawl（4 分）

(a) 使用上面路径中的 WARC 副本，查看其中第一个页面。它经过 gzip 压缩，可运行：zcat /shared-data/CC/example.warc.gz | less 。less 支持方向键、Page Up、Page Down，按 q 退出。

第一个网页的 URL 是什么？现在是否仍可访问？仅通过原始 HTML 能否看出页面主题？

提交内容：用 2-3 句话回答。

(b) 查看对应 WET：zcat /shared-data/CC/example.warc.wet.gz | less 。WET 还含 Content-Length 等 HTTP 头，这些不是提取的正文。第一条示例应包含刚才 HTML 中提取出的文本。

很多文本反映的是 HTML 结构，并非页面主要内容。哪些部分本应被提取器过滤？把这种文本作为训练数据可能出什么问题？反过来，模型又可能从中学到什么有用信息？

提交内容：用 3-4 句话回答。

(c) 什么是好的训练样本，很大程度上取决于使用场景。描述一个该示例适合进入训练集的应用领域，再描述一个不适合的领域。

提交内容：用 1-2 句话回答。

(d) 再查看 25 条 WET 记录。简要记录每条的语言（若能识别）、域名、页面类型等。看了多少条才遇到你认为的“高质量”网页？

提交内容：25 份文档的简短标注，包含语言、域名、页面类型及其他观察，并报告首次遇到高质量样本前查看了多少条。

2.2 HTML 转文本

从 WARC/WET 可以看到，HTML 文本提取并不容易。提取程序通常寻找 HTML 中可见内容，例如应该承载正文段落的 <p> 标签。但这些可见内容往往远多于在浏览器里看到的主体。例如 StackOverflow 的主体是问题与答案，但菜单、其他 StackExchange 页面的无关链接、页脚、注册登录链接也都是可见文本，难以可靠区分。

有许多工具实现了文本提取流水线。本作业使用 Resiliparse 库，还用它解决更基础的问题：检测原始内容字节的文本编码。尽管原文援引 Wikipedia 称 98.2% 的网页使用 UTF-8，流水线仍需稳健支持其他编码。[脚注 2：https://resiliparse.chatnoir.eu/en/stable/index.html]

提示：建议使用 FastWARC 遍历每个 WARC 文件中的记录，尤其可用 from fastwarc.warc import ArchiveIterator, WarcRecordType 。文档：https://resiliparse.chatnoir.eu/en/stable/man/fastwarc.html 。

题目（extract_text）：HTML 转文本（3 分）

(a) 编写函数，从包含原始 HTML 的字节串中提取文本，使用 resiliparse.extract.html2text.extract_plain_text。此函数需要字符串，因此先把字节解码为 Unicode 字符串。输入不一定是 UTF-8，UTF-8 解码失败时应检测编码。resiliparse.parse.encoding.detect_encoding() 可能有帮助。

提交内容：接收 HTML 字节串、返回提取文本字符串的函数。实现 run_extract_text_from_html_bytes，确保通过 uv run pytest -k test_extract_text_from_html_bytes。

(b) 在单个 WARC 文件上运行提取函数，与对应 WET 中的提取结果比较。有哪些相似和不同？哪个更好？

提交内容：用 2-3 句话比较两种提取结果。

2.3 语言识别

网上有数千种语言，但在多数计算预算下，要训练能有效利用如此多样数据的多语模型并不容易。因此，许多源自 CC 的训练集只包含有限几种语言。

fastText（https://fasttext.cc）提供高效文本分类器，既可训练自定义分类器，也提供预训练模型，包括语言识别模型。lid.176.bin 可从 https://fasttext.cc/docs/en/language-identification.html 下载，也位于 /shared-data/classifiers/lid.176.bin。

语言过滤器通常根据分类器得分决定保留页面。使用 fastText 实现语言识别过滤器，给出预测及非负置信度。

题目（language_identification）：语言识别（6 分）

(a) 编写函数，接收 Unicode 字符串，识别其中主要语言，返回二元组：语言标识与 0 到 1 的置信度。

提交内容：给出最可能语言和得分的函数。实现 run_identify_language，通过 uv run pytest -k test_identify_language 中两项测试。测试假定英语标识为 "en"、中文为 "zh"，必要时在适配器中映射。

(b) 语言模型的推理行为很大程度取决于训练数据，过滤流水线的问题会传导到下游。语言识别出错可能引发什么问题？在面向用户产品等风险更高的场景中，如何缓解？

提交内容：用 2-5 句话回答。

(c) 对之前从 WARC 提取的文本运行语言识别。随机选 20 个例子人工标注语言，与分类器预测比较，报告错误。英语文档占多少？根据观察，适合的置信度过滤阈值是多少？

提交内容：用 2-5 句话回答。

2.4 个人身份信息

网页中有大量可联系或识别个人的信息，如邮箱、电话号码、IP 地址。我们可能不希望面向用户的模型输出真实人物的这类信息，因此常在训练集中将其遮蔽。下面分别实现邮箱、电话、IP 地址的遮蔽程序。

题目（mask_pii）：个人身份信息（3 分）

(a) 编写邮箱遮蔽函数，接收字符串，将全部邮箱地址替换为 "|||EMAIL_ADDRESS|||"。可查找可靠的检测正则表达式。

提交内容：返回新字符串与替换次数二元组的函数。实现 run_mask_emails，通过 uv run pytest -k test_mask_emails 的所有测试。

(b) 编写电话遮蔽函数，将电话号码替换为 "|||PHONE_NUMBER|||"。电话格式十分多样，要完全可靠很困难；至少应覆盖美国最常见格式，并容忍轻微格式差异。

提交内容：返回新字符串与替换次数二元组的函数。实现 run_mask_phone_numbers，通过 uv run pytest -k test_mask_phones。

(c) 编写 IP 遮蔽函数，本题只需处理 IPv4，即以点分隔、每段不超过 255 的四个数。将地址替换为 "|||IP_ADDRESS|||"。

提交内容：返回新字符串与替换次数二元组的函数。实现 run_mask_ips，通过 uv run pytest -k test_mask_ips。

(d) 若在训练集上简单机械地应用这些过滤器，语言模型下游可能出现什么问题？如何缓解？

提交内容：用 2-5 句话回答。

(e) 对之前从 WARC 提取的文本运行 PII 遮蔽。随机查看 20 个发生过替换的样本，给出误报与漏报示例。

提交内容：用 2-5 句话回答。

2.5 有害内容

未过滤的网页转储中有大量我们不希望模型在推理时重复的文本。有时它们甚至来自通常无害的网站，例如 Wikipedia 某些页面的用户评论就很有毒性。虽然几乎不可能给“有害”划出清晰边界，许多流水线仍会过滤主要包含有害内容的页面。

识别方法包括统计禁用词表中的词，或用人工标签训练简单分类器。本部分关注两大类：NSFW（不适合工作场所的内容），包括色情、粗俗用语及其他可能令人不适的内容；有毒言论，即“粗鲁、不尊重他人或不讲道理，可能迫使人离开讨论的语言”。[脚注 3：https://current.withgoogle.com/the-current/toxicity/]

使用 Dolma 项目 [1] 提供的 fastText 预训练模型判断文本是否属于这些类别。分类器用 Jigsaw Toxic Comments 数据集训练，其中包含带多种标签的 Wikipedia 评论。[脚注 4：https://paperswithcode.com/dataset/toxic-comment-classification-challenge]

NSFW 模型下载：https://dolma-artifacts.org/fasttext_models/jigsaw_fasttext_bigrams_20230515/jigsaw_fasttext_bigrams_nsfw_final.bin 。仇恨言论模型下载：https://dolma-artifacts.org/fasttext_models/jigsaw_fasttext_bigrams_20230515/jigsaw_fasttext_bigrams_hatespeech_final.bin 。

共享目录还提供：

• /shared-data/classifiers/dolma_fasttext_hatespeech_jigsaw_model.bin：仇恨与有毒言论分类器。

• /shared-data/classifiers/dolma_fasttext_nsfw_jigsaw_model.bin：NSFW 分类器。

用它们实现函数，接收页面内容的 Unicode 字符串，返回例如 toxic、non-toxic 的标签，以及置信度。

题目（harmful_content）：有害内容（6 分）

(a) 编写检测 NSFW 内容的函数。

提交内容：判断字符串是否含 NSFW，并返回标签和置信度。实现 run_classify_nsfw，通过 uv run pytest -k test_classify_nsfw。测试来自 Jigsaw，仅做基本检查，不能证明分类器准确；准确性需要自行验证。

(b) 编写有毒言论检测函数。

提交内容：判断字符串是否为有毒言论，返回标签与置信度。实现 run_classify_toxic_speech，通过 uv run pytest -k test_classify_toxic_speech。同样，这只是来自 Jigsaw 的基本检查。

(c) 用这些过滤器构造训练集时，下游语言模型可能出现什么问题？如何缓解？

提交内容：用 2-5 句话回答。

(d) 对 WARC 提取文本运行有害内容过滤，随机查看 20 个样本，比较分类器预测与人工判断，报告错误。有害文档占比多少？根据观察，适合的置信度阈值是多少？

提交内容：用 2-5 句话回答。

2.6 质量规则

即使按语言过滤并去除有害内容，剩余页面中仍有许多不适合语言建模的低质量文本。“质量”难以定义，但查看 CC 可发现一些典型情况：付费墙页面、坏链接占位页、登录 / 注册 / 联系表单、主要由图片或视频等非文本内容构成而在文本提取时丢失主体的页面。

Gopher 论文 [2] 提出了一组简单质量过滤器，用易理解的启发式规则去除这类明显不合适的样本。规则涉及文档长度、词长、符号与词的比例，以及某些英语停用词是否出现。本作业实现其中一部分，删除满足以下任一条件的文档：

• 少于 50 个词，或超过 100,000 个词。

• 平均词长不在 3 到 10 个字符范围内。

• 超过 30% 的行以省略号 "..." 结尾。

• 至少含一个字母字符的词所占比例低于 80%。

完整规则见 Gopher 论文附录 A。

题目（gopher_quality_filters）：Gopher 质量过滤器（3 分）

(a) 至少实现上述规则子集。可使用 NLTK 的 nltk.word_tokenize 分词，但不是强制要求。

提交内容：仅接收一个字符串，返回布尔值表示是否通过过滤的函数。实现 run_gopher_quality_filter，通过 uv run pytest -k test_gopher。

(b) 在 WARC 提取文本上运行规则过滤，随机查看 20 个例子，比较过滤结果与人工判断，评论不一致的情况。

提交内容：用 2-5 句话回答。

2.7 质量分类器

现在超越 Gopher 规则所捕捉的简单形式特征。按质量排序内容也是信息检索中的基本问题。搜索引擎的一种经典信号是网页链接结构：高质量页面往往链接到其他高质量页面 [3]。

OpenAI 构建 GPT-2 的 WebText 数据集 [4] 时，采用了相似想法：收集 Reddit 中 karma 分数超过最低阈值的评论所链接的页面。也可用 Wikipedia 代替 Reddit，因为 Wikipedia 引用的外部来源通常较可信 [5]。

控制来源通常能得到高质量内容，但按今天的标准，数据集会偏小：OpenWebText 只有 40GB 文本，The Pile 则超过它的 20 倍。一种做法是把这些参考页面当正例、随机 CC 页面当负例，训练 fastText 分类器，再用其质量分数过滤整个 CC。阈值会在精确率与召回率之间做权衡。

本部分要构建质量分类器。课程组已从较新的 Wikipedia 转储中提取参考页 URL，位于 /shared-data/wiki/enwiki-20260501-extracted_urls.txt.gz，包含截至 2026 年 5 月英语 Wikipedia 页面的外部链接。你应从中抽样，获取分类器的“高质量”正例。正例仍可能含不需要的内容，因此可进一步用语言识别、规则过滤等改善质量。给定 URL 文件，可用 wget 将页面抓取为 WARC：

```
wget --timeout=5 \
    -i subsampled_positive_urls.txt \
    --warc-file=subsampled_positive_urls.warc \
    -O /dev/null
```

原文第 9 页

题目（quality_classifier）：质量分类器（15 分）

(a) 训练分类器，接收文本并输出数值质量分数。

提交内容：供下一小问使用的质量分类器。

(b) 编写函数，判断页面质量高低并返回标签置信度。

提交内容：只接收字符串，返回“是否高质量”的标签与置信度二元组。实现 run_classify_quality，运行 uv run pytest -k test_classify_quality，至少正确分类提供的两个样本。

3 去重

互联网上有大量重复内容。有些页面完全相同，例如归档副本，或常见 Web 服务器生成的默认 404 页面。但更多重复发生在细粒度层面。例如 Stack Overflow 的各问题页有独特的问题、评论和答案，同时又共享完全相同的页头、菜单、页脚。本节先处理精确重复，再处理近似重复。

3.1 精确行去重

一种简单方法是仅保留文档中在整个语料里只出现一次的行。这足以消除页头、菜单等大量冗余。简单情况下，删除所有在其他地方完全重复的行后，便能保留每页独有的主体，例如 StackOverflow 的问答。

先遍历一遍语料，统计每行出现次数；第二遍重写文档，仅保留唯一行。直接保存计数键可能需要与全部不同文本行相当的空间。节约内存的简单技巧是用行的哈希作键，使键的大小固定，不依赖行长。下面实现此方法。

题目（exact_deduplication）：精确行去重（3 分）

函数接收输入文件路径列表，先用哈希节约内存地统计语料中每行的频次，再重写各文件，仅保留唯一行。

提交内容：接收 (a) 输入文件路径列表、(b) 输出目录的函数。把每个输入文件以同名写到输出目录，删除在全部输入文件中出现超过一次的行。例如输入 a/1.txt、a/2.txt，输出目录 b/，则写 b/1.txt、b/2.txt。实现 run_exact_line_deduplication，通过 uv run pytest -k test_exact_line_deduplication。

3.2 MinHash + LSH 文档去重

精确去重可删除逐字相同的内容，但无法处理细微差异。例如软件许可证通常由模板加年份、作者名生成，不同 MIT 许可项目的文本大部分相同，却并非完全一致。要移除这类主要由模板组成的重复内容，需要模糊去重。我们将结合 MinHash 与局部敏感哈希（LSH）高效进行文档级模糊去重。

[脚注 5] 详细介绍见 J. Leskovec、A. Rajaraman、J. D. Ullman [6] 第 3 章：http://infolab.stanford.edu/~ullman/mmds/ch3n.pdf 。

文档相似性采用各文档 n-gram 集合的 Jaccard 相似度，定义为 |S∩T|/|S∪T|。朴素方法是把每个文档表示为 n-gram 集合，计算所有文档对的相似度，超过阈值就标为重复。但对 CC 等大型集合不可行；直接保存 n-gram 集合也比文档本身占用更多内存。

MinHash

用紧凑“签名”替代完整 n-gram 集合，希望比较两份签名就能近似原集合的 Jaccard 相似度。MinHash 签名满足这一性质。

给定 S={s_1,…,s_n}，使用 k 个不同哈希函数 h_1,…,h_k，每个将 n-gram 映射到整数。对函数 h_i，minhash(h_i,S)=min(h_i(s_1),…,h_i(s_n))。文档签名是长度 k 的向量：[minhash(h_1,S),…,minhash(h_k,S)]。

[脚注 6] k 个函数可以来自同一哈希族、使用不同种子。例如 MurmurHash3 是一个哈希函数族，具体种子确定族内的具体函数。

两个文档的 n-gram 集合 S_1、S_2，其 Jaccard 相似度可用“签名中对应位置的 MinHash 值相同的比例”近似，证明见 [6] 第 3 章 3.3.3 节。例如签名 [1,2,3,2] 和 [5,2,3,4] 的第 2、3 个位置相同，估计相似度为 2/4。

局部敏感哈希（LSH）

MinHash 节省了表示空间，并在期望意义上保留相似度，但若仍比较全部文档对，开销依然很大。LSH 能高效把可能高度相似的文档放入同一桶。

将长度 k 的签名分成 b 个 band（分段），每段 r 个值，满足 k=br。例如 100 维签名可以拆成 2×50、4×25、50×2 等。两个文档若某个对应 band 的全部值相同，就进入该 band 的同一桶，成为候选重复文档。因此，在签名长度固定时，增加 band 数会提高召回率、降低精确率。

例如 D_1 签名为 [1,2,3,4,5,6]，D_2 为 [1,2,3,5,1,2]。取 3 段、每段 2 个值，D_1 的三个 band 为 [1,2]、[3,4]、[5,6]；D_2 为 [1,2]、[3,5]、[1,2]。第一段相同，所以两者在第一段进入同桶；其他对应段不同，不会同桶。只要至少一个 band 同桶，就属于候选重复，其他 band 是否匹配并不影响候选资格。

找到候选后，可计算候选文档对的真实 Jaccard 相似度，超过阈值则标为重复。

最后跨桶合并重复簇。例如 A、B 在某桶匹配且真实相似度超过阈值；B、C 在另一桶匹配且也超过阈值，则把 A、B、C 视为同一簇。每簇随机保留一份，删除其余文档。

题目（minhash_deduplication）：MinHash + LSH 文档去重（8 分）

编写函数，接收输入文件路径列表，用 MinHash 和 LSH 进行模糊文档去重。为各文档计算 MinHash 签名，使用给定 band 数找候选，再计算候选间真实的 n-gram Jaccard 相似度，删除超过阈值的重复。

为提高召回率，按文献 [7]，在计算签名和 / 或相似度前对文本做规范化：转小写、删除标点、规范空白、去除重音，并应用 Unicode NFD 规范化。

提交内容：函数至少接收 (a) 输入文件路径列表、(b) MinHash 的哈希函数数量、(c) LSH 的 band 数、(d) 以词计的 n-gram 长度、(e) 输出目录。可假设哈希数能被 band 数整除。

将各输入文件以同名重写到输出目录，只写入非候选重复文档，和 / 或从重复簇中随机选中保留的文档。例如 a/1.txt、a/2.txt 对应输出 b/1.txt、b/2.txt。实现 run_minhash_deduplication，通过 uv run pytest -k test_minhash_deduplication。

4 过滤语言建模数据

现在组合已实现的原语，生成语言建模训练数据。

从一批已做英语过滤的 CC WET 文件开始：仅保留 fastText lid.176.bin 预测英语概率至少 70% 的文档，文件位于 /shared-data/english-wet-data。

目标是过滤 CC 转储，使训练出的 Transformer 在 Paloma 基准 [8] 的 C4 100 domains 子集上获得尽可能低的验证困惑度。不要修改模型架构或训练流程，因为本题目标是构造最佳数据。

此验证集包含 C4 语言模型数据集 [9] 中最常见 100 个域名的样本。副本位于 /shared-data/tokenized_paloma_c4_100_domains_validation.bin，已用 GPT-2 tokenizer 分词。可查看数据建立直觉，加载方式如下：

```
import numpy as np
data = np.fromfile(
    "/shared-data/tokenized_paloma_c4_100_domains_validation.bin",
    dtype=np.uint16
)

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")
print(tokenizer.decode(data[0:2000]))
```

原文第 12 页

使用过滤后的数据训练一个 GPT-2 small 形状的模型，再评估 C4 100 困惑度。

允许使用 Paloma 验证数据来构造处理 CC WET 的过滤器或分类器，但绝不能把验证数据直接复制进训练数据。语言模型不应在训练中看到验证集数据。

这批 WET 也相当大，建议尽量用多进程处理，Python concurrent.futures 或 multiprocessing 可能很有帮助。下面是用 concurrent.futures 跨多个进程并行运行函数的最小示例：

```python
import concurrent.futures
import os

from tqdm import tqdm

def process_single_wet_file(input_path: str, output_path: str):
    # TODO: read input path, process the input, and write the output to output_path
    return output_path

# Set up the executor
num_cpus = len(os.sched_getaffinity(0))
executor = concurrent.futures.ProcessPoolExecutor(max_workers=num_cpus)
wet_filepaths = ["a.warc.wet.gz", "b.warc.wet.gz", "c.warc.wet.gz"]
output_directory_path = "/path/to/output_directory/"

futures = []
for wet_filepath in wet_filepaths:
    # For each warc.wet.gz filepath, submit a job to the executor and get a future back
    wet_filename = str(pathlib.Path(wet_filepath).name)
    future = executor.submit(
        process_single_wet_file,
        wet_filepath,
        os.path.join(output_directory_path, wet_filename)
    )
    # Store the futures
    futures.append(future)

# Iterate over the completed futures as they finish, using a progress bar
# to keep track of progress.
for future in tqdm(
        concurrent.futures.as_completed(futures),
        total=len(wet_filepaths),
):
    output_file = future.result()
    print(f"Output file written: {output_file}")
```

原文第 13 页

对较大的文本块使用 concurrent.futures、multiprocessing 等本地并行；要跨 Modal worker 并行，对输入调用 .map(...)。

建议用 FastWARC 遍历 WET 记录，用 tldextract 从 URL 提取域名以便过滤。可使用以下导入；tldextract 项目为 https://github.com/john-kurkowski/tldextract 。

```python
from fastwarc.warc import ArchiveIterator, WarcRecordType
from tldextract import TLDExtract
```

原文第 13 页

题目（filter_data）：过滤语言建模数据（6 分）

(a) 编写脚本，过滤 /shared-data/english-wet-data 下的 CC WET。可自由使用之前实现的原语，也可探索其他方法，例如按 n-gram 语言模型困惑度过滤。目标是使训练结果在 Paloma 的 C4 100 domains 上困惑度最低。

再次强调，可以用 Paloma 验证数据构造过滤器或分类器，但不得直接复制进训练数据。脚本应报告每个过滤器保留的样本数，以理解各步骤对最终输出的影响。

提交内容：一个脚本或一组脚本，并行过滤提供的 WET 以生成语言建模数据；书面说明各步骤删除的样本分别占全部被删除样本的多少。

(b) 过滤这批 WET（最初为 2,500 个原始 WET 文件）要多久？处理完整 Common Crawl 转储预计要多久？

提交内容：数据过滤流水线的运行时间。

生成数据后，查看样本以理解其内容。

题目（inspect_filtered_data）：检查过滤后的数据（4 分）

(a) 从过滤结果随机取 5 个例子，评价质量及是否适合语言建模，尤其考虑降低 C4 100 domains 困惑度的目标。

提交内容：5 个随机样本，文档过长时只展示相关片段即可；每个用 1-2 句话描述并评价训练价值。

(b) 随机取 5 个被过滤脚本删除和 / 或修改的 CC WET 样本。是哪一步处理的？你认为处理合理吗？

提交内容：原始 WET 中的 5 个随机被丢弃样本，可只展示相关片段；每个用 1-2 句话描述并判断删除是否合理。

(c) 若上述分析提示应修改数据流水线，可以先修改再训练。报告尝试过的调整及数据迭代。

提交内容：尝试的数据修改和 / 或迭代说明。

训练前需要分词。通过 transformers 使用 GPT-2 tokenizer，把过滤文本编码成整数 ID 序列；每篇文档末尾不要忘记添加结束标记 <|endoftext|>。起始代码如下：

```
import multiprocessing

import numpy as np
from tqdm import tqdm
```

原文第 14 页

```python
from transformers import AutoTokenizer

input_path = "path/to/your/filtered/data"
output_path = "path/to/your/tokenized/data"

tokenizer = AutoTokenizer.from_pretrained("gpt2")

def tokenize_line_and_add_eos(line):
    return tokenizer.encode(line) + [tokenizer.eos_token_id]

with open(input_path) as f:
    lines = f.readlines()

pool = multiprocessing.Pool(multiprocessing.cpu_count())
chunksize = 100
results = []
for result in tqdm(
    pool.imap(tokenize_line_and_add_eos, lines, chunksize=chunksize),
    total=len(lines),
    desc="Tokenizing lines"
):
    results.append(result)

pool.close()
pool.join()

# Flatten the list of ids and convert to numpy array
all_ids = [token_id for sublist in results for token_id in sublist]
print(f"Tokenized and encoded {input_path} into {len(all_ids)} tokens")
ids_array = np.array(all_ids, dtype=np.uint16)
ids_array.tofile(output_path)
```

原文第 15 页

题目（tokenize_data）：数据分词（2 分）

编写脚本，对过滤数据分词并序列化。按上面的示例使用 ids_array.tofile(output_path)，其中 ids_array 是 np.uint16 类型的整数 ID NumPy 数组，以兼容提供的训练脚本。

过滤后数据共有多少 token？

提交内容：分词与序列化脚本，以及生成数据的 token 数。

现在可以训练模型。使用 GPT-2 small 形状、约 430M 参数的模型，在生成数据上训练，并定期在 C4 100 domains 上验证。训练使用 8 张 B200、数据并行、每设备批大小 128，共 16,384 次优化步骤。上下文长度为 512，因此训练中采样约 2^{33}=8.6B 个 token。课程组用此设置训练耗时约 2 小时。

译注：原文同时使用“GPT-2 small-shaped”和“约 430M 参数”两种描述，均在此保留；实际配置以提供的训练配置文件为准。

用 scripts/train.py 启动训练；超参数可在 cs336_basics/train_config.py 查看。通过 --train-bin 指定分词后的训练数据：[脚注 7：https://github.com/stanford-cs336/assignment4-data/blob/master/scripts/train.py]

```
uv run modal run scripts/train.py --train-bin /root/data/your_data.bin
```

原文第 16 页

再次强调，本作业优化的是数据，不是通过修改模型或优化过程降低损失，因此不要修改训练流程和训练脚本。

训练后可用下面的命令从保存模型采样：

```
uv run modal run scripts/generate_with_gpt2_tok.py --model-path /root/data/output/your_data
```

原文第 16 页

题目（train_model）：训练模型（8 分）

在分词后的数据集上训练 GPT-2 small 形状、约 430M 参数的语言模型，定期测量 C4 100 domains 验证损失。模型达到的最佳验证损失是多少？

提交内容：记录到的最佳验证损失、对应学习曲线，以及所做工作的说明。

参考文献

保留英文题名以便检索。

[1] L. Soldaini 等. “Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research.” arXiv:2402.00159，2024。（用于语言模型预训练研究的三万亿 token 开放语料。）

[2] J. W. Rae 等. “Scaling language models: Methods, analysis & insights from training gopher.” arXiv:2112.11446，2021。（扩展语言模型：Gopher 训练的方法、分析与洞见。）

[3] L. Page, S. Brin, R. Motwani, T. Winograd. “The pagerank citation ranking: Bring order to the web.” 第 7 届 International World Wide Web Conference 论文集，1998。（PageRank 引用排序：为网络建立秩序。）

[4] A. Radford 等. “Language models are unsupervised multitask learners.” OpenAI blog，卷 1，第 8 期，第 9 页，2019。（语言模型是无监督多任务学习器。）

[5] H. Touvron 等. “LLaMA: Open and Efficient Foundation Language Models.” 2023。（开放且高效的基础语言模型。）

[6] J. Leskovec, A. Rajaraman, J. D. Ullman. Mining of Massive Datasets，第 2 版，Cambridge University Press，2014。（《大规模数据挖掘》。）

[7] G. Penedo 等. “The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only.” 2023。（仅靠网页数据超越精选语料。）

[8] I. Magnusson 等. “Paloma: A Benchmark for Evaluating Language Model Fit.” 2023。（评估语言模型拟合能力的基准。）

[9] C. Raffel 等. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” Journal of Machine Learning Research，21(140)，1-67 页，2020。http://jmlr.org/papers/v21/20-074.html（以统一文本到文本 Transformer 探索迁移学习极限。）